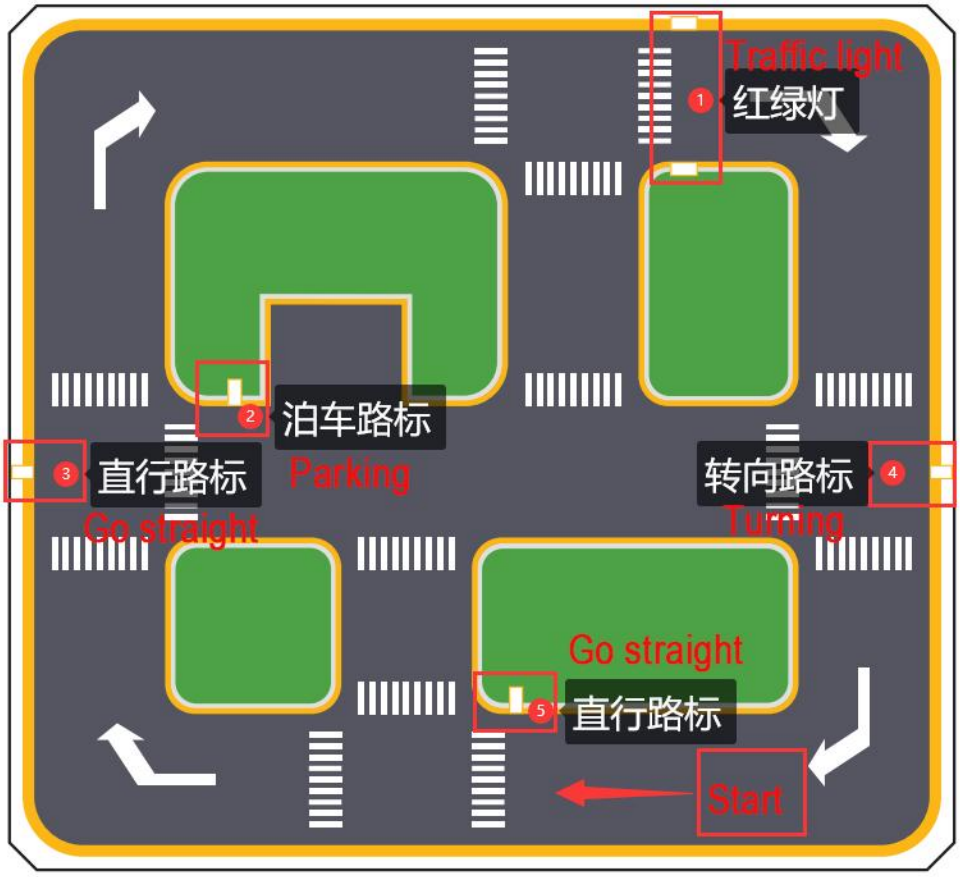
Integrated Application

This lesson provides instructions for implementing comprehensive driverless game on the robot, covering lane keeping, road sign detection, traffic light recognition, steering decision-making, and self-parking.

1. Getting Ready

1.1 Map Setup

To ensure accurate navigation, place the map on a flat, smooth surface, free of wrinkles and obstacles. Position all road signs and traffic lights at designated locations on the map, facing clockwise. The starting point and locations of road signs are indicated below:



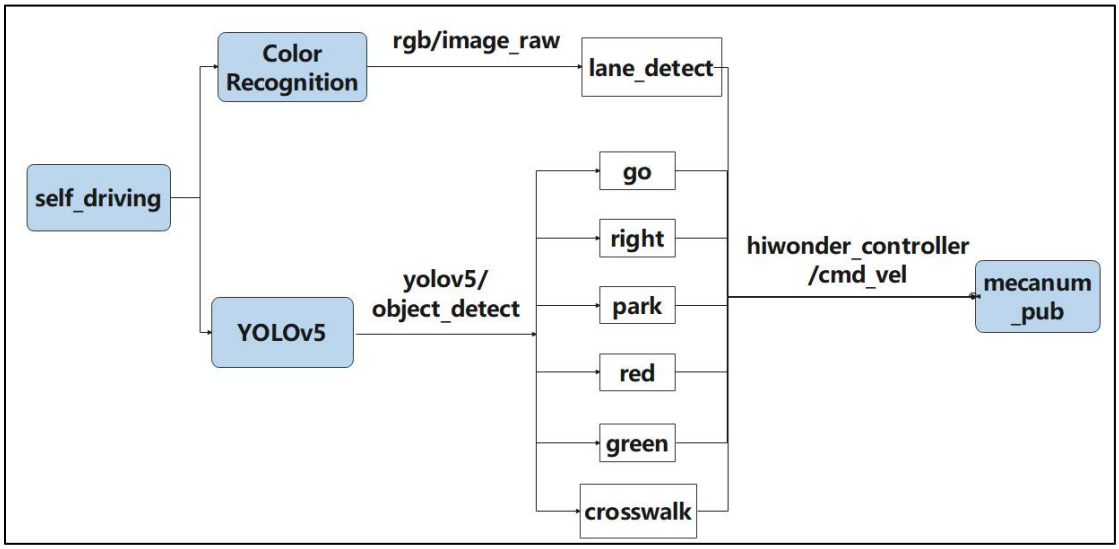
1.2 Color Threshold Adjustment

Due to variations in light sources, it's essential to adjust the color thresholds for

"black, white, red, green, blue, and yellow" based on the guidelines provided in the "14.ROS+OpenCV Lesson" prior to starting.

If the robot encounter inaccurate recognition while moving forward, readjust the color threshold specifically in the map area where recognition fails.

2. Program Logic



Actions implemented so far include:

1. Following the yellow line in the outermost circle of the patrol map.
2. Slowing down and passing if a zebra crossing is detected.
3. Making a turn upon detection of a turn sign.
4. Parking the vehicle and entering the parking lot upon detection of a stop sign.
5. Halting when a red light is detected.
6. Slowing down when passing a detected street light.

First, load the model file trained by YOLOv5 and the required library files, obtain real-time camera images, and perform operations such as erosion and dilation on the images.

Next, identify the target color line segment in the image and gather information such as size and center point of the target image. Then, invoke the model through YOLOv5 and compare it with the target screen image.

Finally, adjust the forward direction based on the offset comparison of the target image's center point to keep the robot in the middle of the road. Additionally, perform corresponding actions based on different recognized landmark information during map traversal.

3. Operation Steps

Note: the input command should be case sensitive, and keywords can be implemented using Tab key.

1) Start the robot, and access the robot system desktop using VNC.

2) Click-on  to open the command-line terminal.

3) Execute the command to disable the app auto-start service.

~/.stop_ros.sh

```
~/.stop_ros.sh
```

4) Type the command in the command line terminal and press “Enter”.

ros2 launch example self_driving.launch.py

```
ros2 launch example self_driving.launch.py
```

5) Open a new command line terminal. Execute the command to open the interface for live camera feed.

rqt

```
rqt
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '
tmp/runtime-ubuntu'
```

6) If you want to terminate the game, access the relevant terminal window and press “Ctrl+C”.

```
yolov5_node-10] [INFO] [0000001844.050502112] [yolov5]: 0.75
apply_calib-2] [INFO] [0000001844.093072896] [imu_calib]: Gyro calibration complete! (bias = [-0.100, 0.038, -0.055])
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] Loaded engine size: 15 MiB
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [W] Using an engine plan file across different models of devices is not recommended and
is likely to affect performance or even cause errors.
ERROR] [self_driving-11]: process has died [pid 6586, exit code 1, cmd '/home/ubuntu/ros2_ws/install/example/lib/example/self_drivi
g --ros-args --params-file /tmp/launch_params_3zms_3zj --params-file /tmp/launch_params_elvpyfqa'].
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] Init cuBLAS/cuBLASLt: CPU +5, GPU +9, now: CPU 105, GPU 5186 (MiB)
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] Init cuDNN: CPU +1, GPU +12, now: CPU 106, GPU 5198 (MiB)
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] TensorRT-managed allocation in engine deserialization: CPU +0, GPU
+13, now: CPU 0, GPU 13 (MiB)
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] Init cuBLAS/cuBLASLt: CPU +0, GPU +5, now: CPU 91, GPU 5184 (MiB)
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] Init cuDNN: CPU +0, GPU +3, now: CPU 91, GPU 5187 (MiB)
yolov5_node-10] [01/01/1970-08:30:44] [TRT] [I] [MemUsageChange] TensorRT-managed allocation in IExecutionContext creation: CPU +0,
GPU +16, now: CPU 0, GPU 29 (MiB)
yolov5_node-10] [INFO] [0000001844.512830336] [yolov5]: start
```

Once you've completed the game experience, you can initiate the app service by following the instructions or restarting the robot.

If the app service is not activated, the associated app functions will be disabled. (The app service will automatically start when the robot restarts).

To restart the app service, enter the following command and wait for the buzzer to emit a single beep, indicating that the service startup is complete.

```
sudo systemctl restart start_node.service
```

4. Program Outcome

◆ **Lane Keeping**

Upon initiating the game, the robot will track the line and identify the yellow line at the road's edge. It will execute forward and turning actions based on the straightness or curvature of the yellow line to maintain lane position.

◆ **Traffic Light Recognition**

When the car encounters a traffic light, it will halt if the light is red and proceed if it's green. Upon approaching a zebra crossing, the car will automatically decelerate and proceed cautiously.

◆ **Turn and Parking Signs**

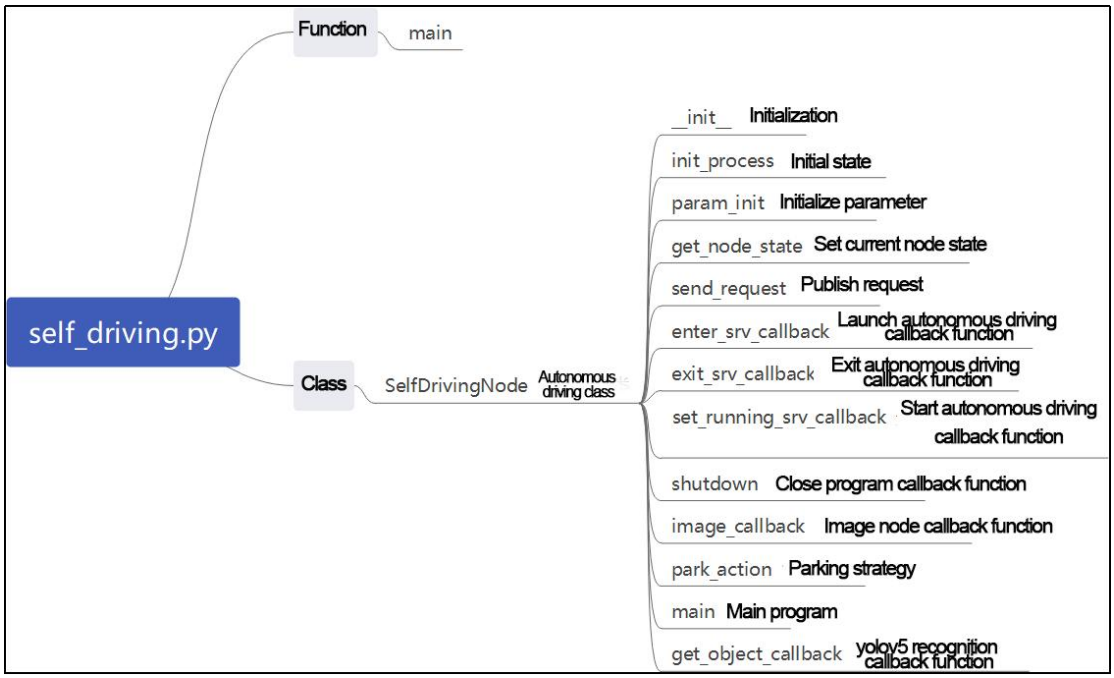
Upon detecting traffic signs while moving forward, the car will respond accordingly. If it encounters a right turn sign, it will execute a right turn and continue forward. In the case of a parking sign, it will execute a parking maneuver.

Following these rules, the robot will continuously progress forward within the map.

5. Program Analysis

The source code for this program can be found at:

/home/ubuntu/ros2_ws/src/example/example/self_driving/self_driving.py



Function:

```
def main():
    node = SelfDrivingNode('self_driving')
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    executor.spin()
    node.destroy_node()
```

Start the autonomous driving class.

Class:

```
    def __init__(self, name):
        rclpy.init()
        super().__init__(name, allow_undeclared_parameters=True, automatically_declare
_parameters_from_overrides=True)
        self.name = name
        self.is_running = True
        self.pid = pid.PID(0.4, 0.0, 0.05)
        self.param_init()

```

init：

```python
class SelfDrivingNode(Node):
    def __init__(self, name):
        rclpy.init()
        super().__init__(name, allow_undeclared_parameters=True, automatically_declare_parameters_from_overrides=True)
        self.name = name
        self.is_running = True
        self.pid = pid.PID(0.4, 0.0, 0.05)
        self.param_init()

        self.fps = fps.FPS()
        self.image_queue = queue.Queue(maxsize=2)
        self.classes = ['go', 'right', 'park', 'red', 'green', 'crosswalk']
        self.display = True
        self.bridge = CvBridge()
        self.lock = threading.RLock()
        self.colors = common.Colors()
        # signal.signal(signal.SIGINT, self.shutdown)
        self.machine_type = os.environ.get('MACHINE_TYPE')
        self.lane_detect = lane_detect.LaneDetector("yellow")

        self.mecanum_pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)
        self.joints_pub = self.create_publisher(ServosPosition, '/servo_controller', 1) # 舵机控制(servo control)
        self.servo_state_pub = self.create_publisher(SetPWMServoState, 'ros_robot_controller/pwm_servo/set_state', 1)
        self.result_publisher = self.create_publisher(Image, '~/image_result', 1)

        self.create_service(Trigger, '~/enter', self.enter_srv_callback) # 进入玩法(enter the game)
        self.create_service(Trigger, '~/exit', self.exit_srv_callback) # 退出玩法(exit the game)
        self.create_service(SetBool, '~/set_running', self.set_running_srv_callback)
        # self.heart = Heart(self.name + '/heartbeat', 5, lambda _: self.exit_srv_callback(None))
        timer_cb_group = ReentrantCallbackGroup()
        self.client = self.create_client(Trigger, '/controller_manager/init_finish')
        self.client.wait_for_service()
```

Initialize the required parameters to get the current robot type. Set the color for line following to yellow. Start the chassis control, servo control, and image reading. Set up three services for entering, exiting, and starting. Read the YOLOv5 node.

init_process：

```python
    def init_process(self):
        self.timer.cancel()

        self.mecanum_pub.publish(Twist())
        if not self.get_parameter('only_line_follow').value:
            self.send_request(self.start_yolov5_client, Trigger.Request())
        if self.machine_type != 'JetRover_Tank':
            set_servo_position(self.joints_pub, 1, ((10, 500), (5, 500), (4, 240), (3, 0), (2, 750), (1, 500)))  # 初始姿态(initial posture)
        else:
            set_servo_position(self.joints_pub, 1, ((10, 500), (5, 500), (4, 230), (3, 0), (2, 750), (1, 500)))  # 初始姿态(initial posture)
        time.sleep(1)

        if 1:#self.get_parameter('start').value:
            self.display = True
            self.enter_srv_callback(Trigger.Request(), Trigger.Response())
            request = SetBool.Request()
            request.data = True
            self.set_running_srv_callback(request, SetBool.Response())
```

Initialize the current servo and start the main function.

param_init：

```python
    def param_init(self):
        self.start = False
        self.enter = False
        self.right = True

        self.have_turn_right = False
        self.detect_turn_right = False
        self.detect_far_lane = False
        self.park_x = -1  # 停车标识的x像素坐标(obtain the x-pixel coordinate of a parking sign)

        self.start_turn_time_stamp = 0
        self.count_turn = 0
        self.start_turn = False  # 开始转弯(start to turn)
```

Initialize the parameters needed for position recognition or other operations.

get_node_state：

```python
    def get_node_state(self, request, response):
        response.success = True
        return response
```

Get the current node status.

send_request：

```
140     def send_request(self, client, msg):
141         future = client.call_async(msg)
142         while rclpy.ok():
143             if future.done() and future.result():
144                 return future.result()
```

Used to publish service requests.

enter_srv_callback：

```
146     def enter_srv_callback(self, request, response):
147         self.get_logger().info('\033[1;32m%s\033[0m' % "self driving enter")
148         with self.lock:
149             self.start = False
150             camera = 'depth_cam'#self.get_parameter('depth_camera_name').value
151             self.create_subscription(Image, '/ascamera/camera_publisher/rgb0/image' ,
    self.image_callback, 1)
152             self.create_subscription(ObjectsInfo, '/yolov5_ros2/object_detect', self.g
    et_object_callback, 1)
153             self.mecanum_pub.publish(Twist())
154             self.enter = True
155         response.success = True
156         response.message = "enter"
157         return response
158
```

Enter the autonomous driving game service. Start reading images and YOLOv5 recognition content. Initialize the speed.

exit_srv_callback：

```
159     def exit_srv_callback(self, request, response):
160         self.get_logger().info('\033[1;32m%s\033[0m' % "self driving exit")
161         with self.lock:
162             try:
163                 if self.image_sub is not None:
164                     self.image_sub.unregister()
165                 if self.object_sub is not None:
166                     self.object_sub.unregister()
167             except Exception as e:
168                 self.get_logger().info('\033[1;32m%s\033[0m' % str(e))
169             self.mecanum_pub.publish(Twist())
170         self.param_init()
171         response.success = True
172         response.message = "exit"
173         return response
174
```

Exit the autonomous driving game service. It will stop reading images and YOLOv5 recognition content. Initialize the speed, and reset the parameters.

set_running_srv_callback：

```
175     def set_running_srv_callback(self, request, response):
176         self.get_logger().info('\033[1;32m%s\033[0m' % "set_running")
177         with self.lock:
178             self.start = request.data
179             if not self.start:
180                 self.mecanum_pub.publish(Twist())
181         response.success = True
182         response.message = "set_running"
183         return response
```

Enable the autonomous driving game. Set the “start” parameter to True.

Shutdown：

```
    def shutdown(self, signum, frame):  # ctrl+c关闭处理(press 'ctrl+c' to close the program)
        self.is_running = False
```

The callback function after the program is closed is used to stop the current running program.

image_callback：

```
    def image_callback(self, ros_image):  # 目标检查回调(callback target checking)
        cv_image = self.bridge.imgmsg_to_cv2(ros_image, "rgb8")
        rgb_image = np.array(cv_image, dtype=np.uint8)
        if self.image_queue.full():
            # 如果队列已满，丢弃最旧的图像(if the queue is full, remove the oldest image)
            self.image_queue.get()
        # 将图像放入队列(put the image into the queue)
        self.image_queue.put(rgb_image)
```

Image callback function, which puts the image into the queue and discards expired images.

park_action：

```
    def park_action(self):
        if self.machine_type == 'MentorPi_Mecanum':
            twist = Twist()
            twist.linear.y = -0.2
            self.mecanum_pub.publish(twist)
            time.sleep(0.38/0.2)
        elif self.machine_type == 'MentorPi_Acker':
            twist = Twist()
            twist.linear.x = 0.15
            twist.angular.z = twist.linear.x*math.tan(-0.5061)/0.145
            self.mecanum_pub.publish(twist)
            time.sleep(3)

            twist = Twist()
            twist.linear.x = 0.15
            twist.angular.z = -twist.linear.x*math.tan(-0.5061)/0.145
            self.mecanum_pub.publish(twist)
            time.sleep(2)

            twist = Twist()
            twist.linear.x = -0.15
            twist.angular.z = twist.linear.x*math.tan(-0.5061)/0.145
            self.mecanum_pub.publish(twist)
            time.sleep(1.5)

            set_servo_position(self.joints_pub, 0.1, ((9, 500), ))
```

Parking logic, which will run two different parking strategies based on two different chassis types.

get_object_callback：

```
    # 获取目标检测结果(Obtain the target detection result)
    def get_object_callback(self, msg):
        self.objects_info = msg.objects
        if self.objects_info == []:  # 没有识别到时重置变量(If it is not recognized, reset the variable)
            self.traffic_signs_status = None
            self.crosswalk_distance = 0
        else:
            min_distance = 0
            for i in self.objects_info:
                class_name = i.class_name
                center = (int((i.box[0] + i.box[2])/2), int((i.box[1] + i.box[3])/2))

                if class_name == 'crosswalk':
                    if center[1] > min_distance:  # 获取最近的人行道y轴像素坐标(Obtain recent y-axis pixel coordinate of the crosswalk)
                        min_distance = center[1]
                elif class_name == 'right':  # 获取右转标识(obtain the right turning sign)
                    self.count_right += 1
                    self.count_right_miss = 0
                    if self.count_right >= 5:  # 检测到多次就将右转标志至真(If it is detected multiple times, take the right turning sign to true)
                        self.turn_right = True
                        self.count_right = 0
                elif class_name == 'park':  # 获取停车标识中心坐标(obtain the center coordinate of the parking sign)
                    self.park_x = center[0]
                elif class_name == 'red' or class_name == 'green':  # 获取红绿灯状态(obtain the status of the traffic light)
                    self.traffic_signs_status = i


            self.get_logger().info('\033[1;32m%s\033[0m' % class_name)
            self.crosswalk_distance = min_distance
```

Callback function for reading YOLOv5, which gets the current recognized category.

Main：

```
    def main(self):
        while self.is_running:
            time_start = time.time()
            try:
                image = self.image_queue.get(block=True, timeout=1)
            except queue.Empty:
                if not self.is_running:
                    break
                else:
                    continue
            result_image = image.copy()
            if self.start:
                h, w = image.shape[:2]

                self.get_logger().info('\033[1;33m%s\033[0m' %self.start_slow_down)
                # 获取车道板的二值化图(obtain the binary image of the lane)
                binary_image = self.lane_detect.get_binary(image)
                # self.start_slow_down = True  # 减速标识(sign for slowing down)
                self.get_logger().info('\033[1;33m%s\033[0m' % self.crosswalk_distance)
                # 检测到斑马线,开启减速标志(if detecting the zebra crossing, start to slow down)
                if 70 < self.crosswalk_distance and not self.start_slow_down:  # 只有足够近时才开始减速(The robot starts to slow down only when it is close enough to the zebra crossing)
                    self.count_crosswalk += 1
                    if self.count_crosswalk == 3:  # 多次判断，防止误检测(judge multiple times to prevent false detection)
                        self.count_crosswalk = 0
                        self.start_slow_down = True  # 减速标识(sign for slowing down)
                        self.count_slow_down = time.time()  # 减速固定时间(fixing time for slowing down)
                else:  # 需要连续检测，否则重置(need to detect continuously, otherwise reset)
                    self.count_crosswalk = 0
```

The main function inside the class, which will run different line-following strategies based on different chassis types.

6. FAQ

1. The robot exhibits inconsistent performance during line patrolling, often veering off course.

Adjust the color threshold to better suit the lighting conditions of the actual scene. For precise instructions on color threshold adjustment, please consult "14. ROS+OpenCV Lesson" for detailed guidance.

2. Modify the line patrol processing code

Navigate to the game program path by entering the command.

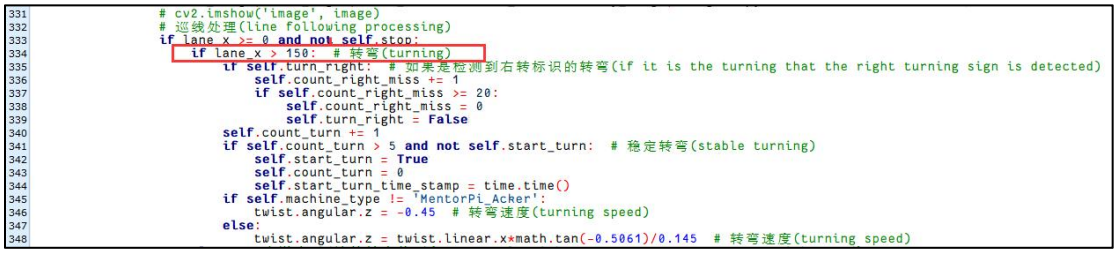
```
cd ~/ros2_ws/src/example/example/self_driving/
```

Execute the command to open the game program.

```
vim self_driving.py
```

```
vim self_driving.py
```

The red box denotes the lane's center point, which can be adjusted to fine-tune the turning effect. Decreasing the value will result in earlier turns, while increasing it will cause later turns.



```
            # cv2.imshow('image', image)
            # 巡线处理(line following processing)
            if lane_x >= 0 and not self.stop:
                if lane_x > 150:  # 转弯(turning)
                    if self.turn_right:  # 如果是检测到右转标识的转弯(if it is the turning that the right turning sign is detected)
                        self.count_right_miss += 1
                        if self.count_right_miss >= 20:
                            self.count_right_miss = 0
                            self.turn_right = False
                    self.count_turn += 1
                    if self.count_turn > 5 and not self.start_turn:  # 稳定转弯(stable turning)
                        self.start_turn = True
                        self.count_turn = 0
                        self.start_turn_time_stamp = time.time()
                    if self.machine_type != 'MentorPi_Acker':
                        twist.angular.z = -0.45  # 转弯速度(turning speed)
                    else:
                        twist.angular.z = twist.linear.x*math.tan(-0.5061)/0.145  # 转弯速度(turning speed)
```

3. The parking location is suboptimal.

You can adjust the parking processing function or modify the starting position of the parking operation.

4. Inaccurate traffic sign recognition.

Adjust the target detection confidence. For detailed instructions, please refer to "3. Operation Steps" for comprehensive learning.